ROCKWELL AUTOMATION + NVIDIA INNOVATION LAB | FALL 2026

ADAPTIVEVISION

Intelligent Robotic Grasping and Sorting for High-Mix, Low-Volume Manufacturing

Theme	Physical AI for the Factory Floor
Project focus	Dataset 3: Robotic Grasping and Sorting
Hackathon duration	October 8 to November 5, 2026
Prepared by	[Student Name]
Team	[Team Name / To Be Confirmed]
Institution	Milwaukee School of Engineering (MSOE)
Document type	Technical Project Plan Draft
Date	October 2026

Project Overview

AdaptiveVision aims to make an industrial robotic sorting cell more adaptable. Today, many pick-and-place programs assume that every part sits at a known, fixed coordinate. We propose a system that uses RGB cameras and computer vision to find objects wherever they are on the worktable, estimate their position and orientation, choose a suitable grasp for the two-finger gripper, and plan a safe robot motion to the correct sorting location. Development will happen first in the provided digital twin, so ideas can be tested without risk to hardware. The team will start with a simple, measurable baseline and add more advanced Physical AI capabilities only after that baseline works. The goal is to show a practical, measured improvement over fixed-coordinate automation.

> **How to read this plan.** Every important statement is labeled by source:
> **CONFIRMED** stated in the event information supplied to the team. **PROPOSED** our team's suggested approach. *TO BE VERIFIED (TBV)* needs confirmation from mentors, Dataset 3 documentation, or the equipment provider.
>
> This is a plan, not a report. Nothing described here has been built, trained, or tested yet, and no results are claimed. Event details were taken from the supplied event information; a public web search on October 9, 2026 did not return a separate official page, so dates should be re-checked on the event portal.

1. Understanding the Innovation Lab

CONFIRMED The Rockwell Automation + NVIDIA Innovation Lab Fall 2026 is a four-week Physical AI hackathon run with MSOE and the University of Wisconsin-Milwaukee's Connected Systems Institute (CSI). Student teams build AI-based solutions to real manufacturing challenges using the CSI testbed. The theme, *Physical AI for the Factory Floor*, asks teams to move AI out of the screen and into machines that work in the physical world.

Physical AI refers to AI systems that perceive a physical environment, interpret what they see, make decisions, and support or perform physical actions through machines or robots. For an industrial robot, this is a repeating loop:

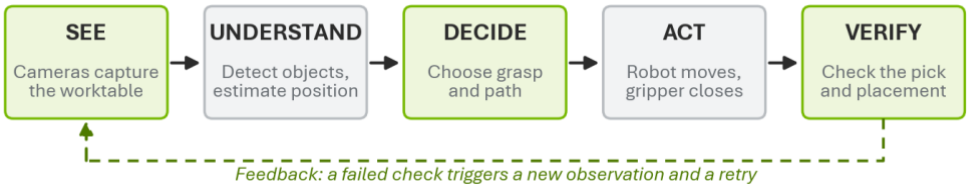


Figure 1. The Physical AI loop applied to a robotic sorting cell (proposed interpretation).

See: cameras capture the worktable. **Understand:** software detects a cube and estimates where it is. **Decide:** the system chooses where to grip it and how to move there. **Act:** the arm moves and the gripper closes. **Verify:** the cameras confirm the cube left the table and arrived in the right tray. If not, the loop starts again.

2. What Is a Manufacturing Testbed?

A **testbed** is a controlled environment where engineers can try new technologies before deploying them in real factories. It matters because it allows controlled experiments, safer testing, repeatable evaluation, validation of new technology against real equipment, and lower risk when a solution later moves to production.

3. What Is HMLV Manufacturing?

High-Mix, Low-Volume (HMLV) manufacturing means a factory makes *many different product variations* (high mix) but *small quantities of each* (low volume). Between production runs, the type, position, or arrangement of parts can change. Traditional fixed-coordinate robot programs work well when products and positions stay predictable. In HMLV settings, every change may require reprogramming, recalibration, or manual setup, and that time adds up. The opportunity is that AI-based perception and adaptable planning can help robots handle some of these variations automatically, without a new program for each change.

4. Factory Context

CONFIRMED The CSI testbed includes a demonstration vial production line. Its process flow is: liquid tanks, fluid mixing, vial filling, cap placement, inspection, robotic handling, and final placement. This kind of fill-cap-inspect process resembles processes used in several manufacturing industries, such as consumer goods, chemicals, and food and beverage. It is background context only. **Dataset 3 uses a separate, standalone FANUC robotic workcell** and is not directly connected to the vial production line.

5. Dataset 3: Robotic Grasping and Sorting

CONFIRMED The challenge is to improve a robotic arm's ability to find, grasp, and relocate objects that may appear in unexpected positions. The provided environment includes a FANUC industrial robotic arm, a pneumatic two-finger gripper, multiple RGB cameras, a digital twin of the arm and environment, and objects such as cubes, caps, and vials.

Component	What it is	What it does	Why it matters / status
FANUC robot	Industrial multi-joint robotic arm	Moves the gripper to precise positions	Executes every pick. Provided; exact model *TBV*
Two-finger gripper	Pneumatic (air-driven) parallel gripper	Opens and closes to hold an object	Grasp must fit its finger width and stroke. Provided
RGB cameras	Standard color cameras, several views	Capture images of the workspace	Only source of object location. Provided; specs *TBV*
Worktable	Surface where objects are placed	Holds objects in varied positions	Defines the pick area. Assumed present, layout *TBV*
Sorting area	Target locations (trays or bins)	Receives sorted objects	Defines success. Destinations *TBV*

Component	What it is	What it does	Why it matters / status
Digital twin	Virtual model of arm and cell	Simulates motion, cameras, objects	Safe testing. Provided; format *TBV*
Robot controller	FANUC control cabinet and software	Runs motion programs, drives joints	Our software must talk to it. Interface *TBV*
AI perception system	Our vision and planning software	Turns images into pick targets	Core contribution. Proposed

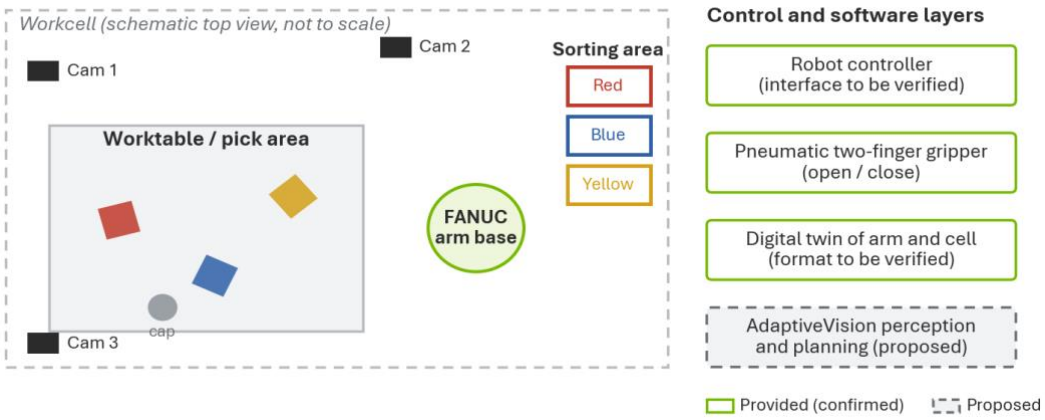


Figure 2. Schematic of the Dataset 3 workcell. Layout, camera placement, and tray positions are illustrative and TBV; this is not a drawing of the real cell.

6. Why Traditional Automation Struggles

A robot is taught to pick a cube from Position A. Someone places the cube at Position B. A fixed-coordinate program still moves to Position A, closes the gripper on nothing, and the task fails until a technician updates the program. An adaptive system closes this gap by looking first: the camera observes the cube, AI detects it, the system estimates its position, a grasp planner chooses a grasp, a motion planner finds a path, the robot attempts the pick, and the system verifies the result.

Aspect	Traditional fixed automation	Vision-guided adaptive automation (proposed)
Object position	Must be exactly where programmed	Measured from camera images every cycle
New part or layout	Reprogram or re-teach points	Update detector classes or sort rules
Failure handling	Often silent; operator notices later	Camera check after each pick, then retry
Setup effort	Low for one fixed job	Higher at first: calibration and model training
Best fit	High-volume, unchanging production	HMLV production with frequent change

7. Project Problem Statement

> *How can a vision-guided robotic system improve the reliability of grasping and sorting objects placed in varying positions and orientations within a high-mix, low-volume manufacturing workcell, compared with a fixed-coordinate pick-and-place program?*

8. Project Objectives

Primary objective: develop and evaluate a vision-guided pick-and-place system that adapts to changes in object position. **Secondary objectives:** detect objects in camera images; estimate object positions; generate suitable grasp targets; plan feasible robot movements; sort objects into assigned destinations; verify task completion; and evaluate performance under varied conditions.

9. How the Proposed System Works

For every object: (1) it is placed on the worktable; (2) the RGB cameras capture the workspace; (3) the vision system identifies it; (4) the system estimates its location and orientation; (5) a grasp

planner picks a gripping position; (6) a motion planner calculates a collision-free path; (7) the robot controller executes the movement; (8) the gripper attempts the grasp; (9) the robot carries the object to its tray; (10) the system checks whether the task succeeded.

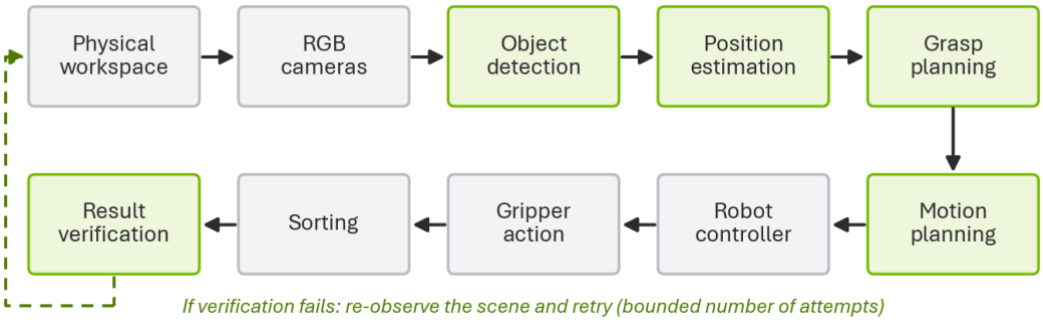


Figure 3. Proposed end-to-end pipeline. Green stages are the main software contributions.

If the object moves before the pick, the plan is based on stale data, so the system re-captures images just before the approach and re-plans if the position changed by more than a set tolerance. **If detection fails** (no object found or low confidence), the robot does not move; the system logs the event and requests a new image or operator help. **If a grasp fails**, the verification check sees the object still on the table and triggers a retry, up to a fixed limit, before flagging the object for an operator.

Concept	Question it answers	Example output
Object detection	What is in the image, and where in pixels?	"Red cube" in a box at pixels (412, 230)
Object localization	Where is it in the real workspace?	x = 0.42 m, y = 0.15 m on the table
Pose estimation	Where is it and how is it rotated?	Position plus a 35 degree rotation
Grasp planning	Where should the fingers close?	Grasp centered on the cube, fingers across its faces
Motion planning	How does the arm get there safely?	A collision-free series of joint positions
Robot control	How is the motion executed?	Controller drives the motors along the path
Task verification	Did it work?	Cube detected in red tray: success

10. Example Scenario (Proposed Demonstration)

Three cubes are sorted by color: red to the red tray, blue to the blue tray, yellow to the yellow tray.

Scenario	Situation	Expected response of the proposed system
A: Expected positions	Cubes sit where a fixed program expects them	Detects all three, confirms positions, sorts them. Both fixed and adaptive systems should succeed; this is the control case.
B: Unexpected positions	Cubes are moved and rotated	Re-detects each cube, computes new positions and grasp angles, sorts correctly. A fixed program would miss.
C: Failed grasp	Blue cube slips out of the gripper	Verification sees the blue cube still on the table, logs a failure, re-plans, and retries within the attempt limit.

11. Technical Terminology Explained

Terms marked *(adv.)* are optional research topics. All other terms are needed to understand or build the baseline.

Term	Meaning, and use in Dataset 3	Term	Meaning, and use in Dataset 3
Physical AI	AI that perceives, decides, and acts in the physical world. *Here:* The whole see-to-verify loop.	**Forward kinematics**	Joint angles in, gripper position out. *Here:* Check where a planned pose puts the gripper.

Term	Meaning, and use in Dataset 3	Term	Meaning, and use in Dataset 3
Industrial automation	Machines and control systems doing production tasks. *Here:* The FANUC cell is an automated system.	**Inverse kinematics**	Desired gripper position in, joint angles out. *Here:* Turn a grasp pose into arm joint angles.
HMLV	Many product types, few of each. *Here:* Why positions and parts change often.	**Grasp planning**	Choosing where and how to hold an object. *Here:* Pick finger placement on a cube or cap.
Manufacturing testbed	Controlled environment for trying new technology. *Here:* The CSI facility.	**Grasp pose**	Gripper position and orientation at closing. *Here:* Above the cube, rotated to match its faces.
Digital twin	Virtual model linked to a real system and its data. *Here:* Test picks before touching hardware.	**Motion planning**	Finding a path that avoids collisions. *Here:* Route from home to cube to tray.
Simulation	Running a model to predict behavior. *Here:* Rehearse thousands of picks quickly.	**Collision detection**	Checking whether geometry overlaps. *Here:* Avoid hitting the table, trays, other parts.
Sim-to-real	Moving results from simulation to real hardware. *Here:* Final, supervised validation step.	**Trajectory**	A path with timing (positions over time). *Here:* What the controller actually executes.
Synthetic data	Computer-generated, auto-labeled training data. *Here:* Rendered images of cubes for the detector.	**Robot controller**	Computer that runs programs and drives motors. *Here:* FANUC controller; interface TBV.
Domain randomization	Randomly varying lighting, color, pose in simulation. *Here:* Helps the detector work on real images.	**End effector**	Tool at the end of the arm. *Here:* The two-finger gripper.
Computer vision	Software that extracts information from images. *Here:* Finding objects on the table.	**Pneumatic gripper**	Gripper powered by compressed air. *Here:* Mostly open/close; force set by air pressure.
RGB camera	Camera recording red, green, blue color (no depth). *Here:* Main sensor; depth must be inferred.	**Open-loop control**	Acting without checking the result. *Here:* Fixed-coordinate picking.
Multi-view vision	Combining images from several cameras. *Here:* Better position estimate, fewer blind spots.	**Closed-loop control**	Acting, measuring, then correcting. *Here:* Pick, verify, retry.
Camera calibration	Measuring lens properties and camera placement. *Here:* Needed to turn pixels into meters.	**Feedback**	Measured information used to adjust actions. *Here:* Camera check after each pick.
Object detection	Finding and labeling objects in an image. *Here:* "Yellow cube at these pixels".	**Reinforcement learning *(adv.)***	Learning by trial, error, and reward. *Here:* Possible grasp-policy research in simulation.
Object localization	Finding an object's position in the world. *Here:* Table coordinates of the cube.	**Imitation learning *(adv.)***	Learning by copying demonstrations. *Here:* Learn picks from recorded examples.
Object pose	Position plus orientation. *Here:* Where the cube is and how it is turned.	**Robot policy *(adv.)***	Function mapping observations to actions. *Here:* Output of RL or imitation learning.
6D pose estimation *(adv.)*	Estimating 3 position and 3 rotation values. *Here:* Needed for vials lying at odd angles.	**Foundation model *(adv.)***	Large model pre-trained on broad data. *Here:* Used as-is or adapted, never trained here.
Coordinate frame	A reference point and set of axes. *Here:* Camera frame, robot base frame, table frame.	**Vision-language model (VLM) *(adv.)***	Model that reads images and text together. *Here:* Describe scene or check result in words.
Coordinate transformation	Converting a point from one frame to another. *Here:* Camera coordinates to robot coordinates.	**Vision-language-action (VLA) *(adv.)***	Model that outputs robot actions from images and text. *Here:* Research comparison only.
Robot kinematics	Math linking joint angles to gripper position. *Here:* Basis of all arm motion.	**In-context learning *(adv.)***	Model adapts from examples in its prompt, no retraining. *Here:* Show example images of a new object type.

12. Proposed Technology Stack

Not every technology below is necessary. The team will use the smallest stack that delivers a working, measurable baseline. Status refers to whether the event provides it, not whether the team has used it.

Technology	Purpose	How we would use it	Priority	Status
Python	Main language	Glue code, vision, planning, scripts	Essential	Proposed
OpenCV	Classic computer vision	Calibration, color filtering, geometry	Essential	Proposed
YOLO (Ultralytics)	Fast object detector	Detect cubes, caps, vials in images	Essential	Proposed
PyTorch	Deep learning framework	Train or fine-tune the detector	Essential	Proposed
NVIDIA Isaac Sim	Robot simulation	Run the digital twin, render images	Essential	TBV
OpenUSD	3D scene description format	Read and edit the twin's scene files	Useful	TBV
Motion planning (e.g., MoveIt 2 or Isaac Sim tools)	Collision-free paths	Plan approach, grasp, place moves	Essential	TBV
ROS 2	Robot messaging middleware	Only if the twin or robot uses it	Useful	TBV
NVIDIA Isaac Lab	Robot learning on Isaac Sim	Only for RL extension	Optional	Proposed
GitHub	Version control	Code, issues, final submission	Essential	Proposed
Docker	Reproducible environment	Same setup on every machine	Useful	Proposed
Experiment tracking	Record runs and metrics	Spreadsheet or tool such as MLflow	Useful	Proposed
NVIDIA Cosmos	World foundation models	Possibly augment synthetic images	Optional	Proposed
VLMs	Image plus text reasoning	In-context checks of new objects	Optional	Proposed
VLAs	End-to-end action models	Literature comparison only	Optional	Proposed
Reinforcement / imitation learning	Learned grasp policies	Research extension in simulation	Optional	Proposed

13. Recommended Baseline Stack

The first version links proven, well-documented tools: **RGB images, then Python + OpenCV, then an object detector, then calibration and position estimation, then grasp target generation, then motion planning, then the digital twin robot, then pick-and-place evaluation.** This is a good first version because every step can be tested on its own, failures are easy to locate, and it produces measurable numbers early. For objects lying flat on a table, the baseline uses *top-down grasps*: the gripper approaches vertically and only needs an x, y position, a fixed height, and one rotation angle. Because the cameras are RGB only, the table height acts as the known depth, so a calibrated camera can convert a pixel into a table position.

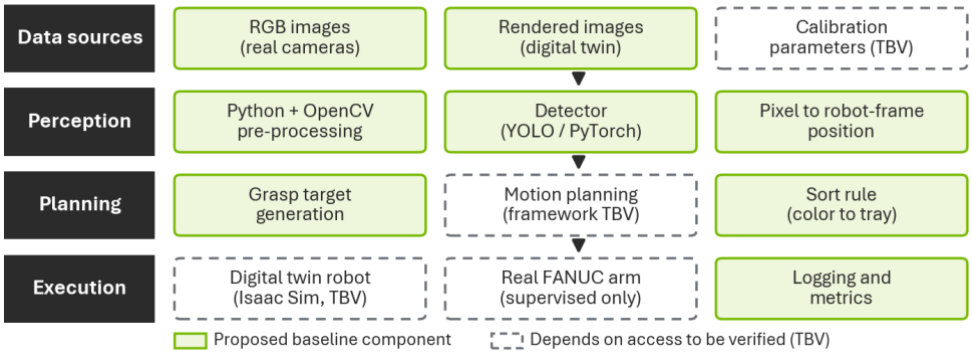


Figure 4. Proposed layered architecture. Dashed boxes depend on interfaces or files still to be verified.

14. Advanced Research Extensions

Extension	What and why	How it could work	Added complexity	Success measure
1. Multi-view localization	Combine cameras to reduce position error and occlusion	Triangulate the same object seen from two or more calibrated cameras	Medium: multi-camera calibration and matching	Lower localization error (mm) than one camera
2. Failure detection and recovery	Catch missed grasps instead of failing silently	Compare before and after images; re-plan and retry	Low to medium	Recovery success rate; fewer unrecovered failures
3. Adaptive grasp selection	Pick the best grasp when objects are rotated, crowded, or a new type	Score candidate grasps by clearance and stability; optional VLM check with example images	Medium to high	Grasp success rate under varied conditions vs. baseline

Training a large robotics foundation model from scratch is out of scope for a four-week hackathon. Any foundation model would be used pre-trained, through prompting or light adaptation.

15. Role of the Digital Twin

A **digital twin** is a virtual model of a real system that mirrors its geometry, behavior, and, ideally, its live data. A *static 3D model* only shows shape; it cannot move or react. A *simulation* runs a model to predict behavior, but it may represent any machine. A digital twin is a simulation tied to one specific real system. For robotics, it lets the team place objects in hundreds of positions, test reachability, generate labeled camera images, and catch collisions or unreachable poses before any real motion. **CONFIRMED** a twin of the arm and environment is provided; *TBV* whether it has a working Isaac Sim control interface.

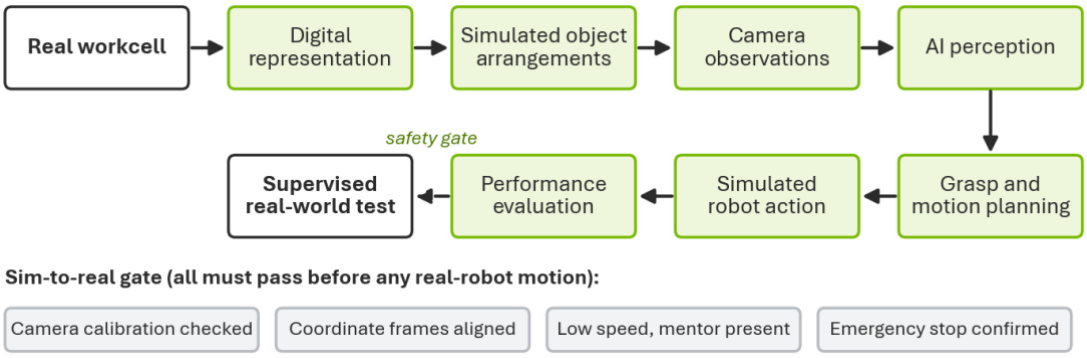


Figure 5. Proposed digital twin to real robot workflow with a sim-to-real safety gate.

16. Synthetic Data Generation

Instead of collecting hundreds of real photographs and labeling each by hand, the digital twin can render images of objects at different positions and record the exact label for each one automatically. Useful variations include object positions, rotations, lighting, camera viewpoints, object combinations, and partial occlusions. These images can train the detector and create test sets with known ground truth. **Limitation:** synthetic images do not perfectly match real lighting, textures, reflections, or physics, so the detector must also be checked on real camera images.

17. Sim-to-Real

Sim-to-real means taking a result that works in simulation and making it work on the physical robot. The main issues are: **camera calibration** (real lens distortion and camera positions must be measured); **robot coordinate frames** (the twin's frames must line up with the real robot base and table); **gripper geometry** (real finger width, stroke, and friction); **simulation fidelity** (how closely simulated physics matches reality); and the **domain gap** (differences between simulated and real images). Real-robot tests will happen only if permitted, at reduced speed, with a mentor present and the emergency stop confirmed.

18. Project Development Plan

CONFIRMED Key dates: **October 8** kickoff; **October 14, 11:59 PM Central Time** team plan deadline; **November 1** final submission deadline; **November 5** final presentations.

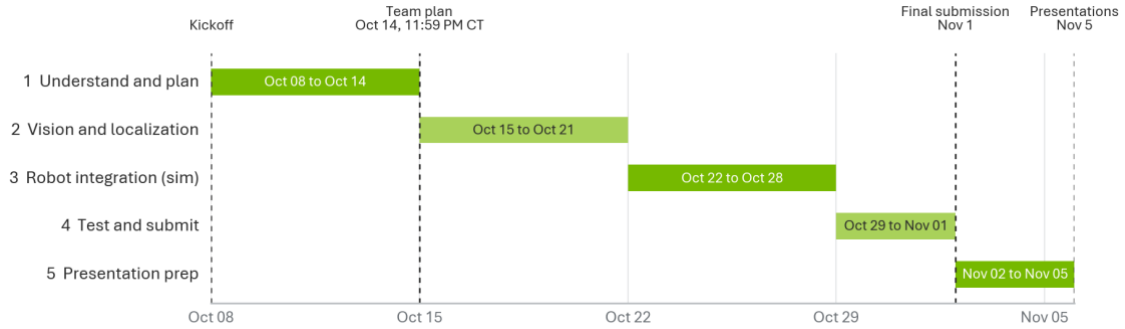


Figure 6. Proposed four-week roadmap with official milestones.

Phase	Main tasks	Deliverable
1. Understand and plan **Oct 8 to 14**	Review Dataset 3 documentation; access the digital twin; inspect camera data; identify robot control interfaces; define scope; select a baseline; prepare the team plan	Project plan and initial architecture
2. Vision and localization **Oct 15 to 21**	Set up environment; inspect images; build a basic detector; study camera calibration; estimate object locations; validate detections	Detection and localization prototype
3. Robot integration **Oct 22 to 28**	Connect perception to the virtual robot; generate grasp targets; integrate motion planning; test pick-and-place; measure grasps; improve reliability	Working simulation prototype
4. Test and submit **Oct 29 to Nov 1**	Test unexpected positions; compare baseline and improved system; measure performance; record video; write one-page summary; organize GitHub; add references; submit	Complete hackathon submission
5. Presentation **Nov 2 to 5**	Refine slides; practice the demo; prepare technical and business explanations; rehearse judge questions	Final presentation on Nov 5

19. Project Evaluation Plan

All tests below are **planned**. Each will run on both the fixed-coordinate baseline and the adaptive system, with the same object layouts, so the comparison is fair: **A** expected positions; **B** unexpected positions; **C** different orientations; **D** multiple objects at once; **E** partially occluded objects; **F** grasp failure and recovery (for example, an object nudged during the approach).

Metric	Meaning	How it will be measured
Object detection accuracy	How often objects are correctly found and labeled	Precision and recall against labeled test images (real and synthetic)
Localization error	Distance between estimated and true position	Millimeters, against twin ground truth or measured real positions
Grasp success rate	Share of grasps that lift and hold the object	Count over all attempts (formula below)
Sorting accuracy	Share of objects placed in the correct tray	Correct placements divided by objects sorted
Task completion time	Time per object, image capture to release	Timestamps in the system log
Collision-free planning rate	Share of planned paths with no collision	Collision checks in simulation
Recovery success rate	Share of failed grasps fixed by a retry	Recovered failures divided by detected failures

Grasp Success Rate = (Successful Grasps / Total Grasp Attempts) x 100. *Illustrative example only, not a result:* if a future test had 46 successful grasps out of 50 attempts, the rate would be (46 / 50) x 100 = 92%.

20. Judging and Award Categories

CONFIRMED Judges represent Rockwell Automation, NVIDIA, MSOE, and UWM. The publicly described expectations are: a solution relevant to the CSI line or robot; a clearly defined manufacturing problem; a working demonstration; measured results; and a business case applicable to manufacturers. The announced award categories, and how this project could address each, are:

- **Most Impactful:** reduce manual reprogramming when parts or positions change, and show the gain with before-and-after measurements.
- **Most Innovative:** a closed loop that sees, verifies, and recovers, unlike a basic fixed-coordinate robot; possibly multi-view localization.
- **Best Use of In-Context Learning:** in-context learning means a model adapts from examples placed in its input, such as a few labeled images of a new part, *without changing its weights*. Fine-tuning, by contrast, retrains the weights. An optional approach is to give a VLM example images of a new object and ask it to identify or check that object. The project will claim this only if it is actually implemented.
- **Most Scalable:** new objects need new detector classes or examples, not a new robot program; the same pipeline could serve other cells.
- **Best Presentation:** a clear problem, a live or recorded demo, honest metrics, and a simple business story.

Detailed judging weights and final award categories were not confirmed in the supplied event materials.

21. Business Impact

Manufacturers might care because a working version of this approach could potentially reduce manual reprogramming, improve production flexibility, handle variable object positions better, reduce downtime caused by missed picks, make sorting more consistent, and simplify adaptation to new product variations. These are potential benefits only; the project will not estimate financial savings without measured data.

22. Official Hackathon Deliverables

Deliverable	Deadline	What it should contain
Team plan	Oct 14, 2026, 11:59 PM CT	Project plan submitted through the event's team plan submission process (form requirements TBV)

Deliverable	Deadline	What it should contain
Demo video (about 5 minutes)	Nov 1, 2026	Problem, environment, AI architecture, working demo, measured results, business value
One-page summary	Nov 1, 2026	Title, problem, solution, stack, implementation, results, business relevance, limitations
GitHub repository	Nov 1, 2026	README, setup steps, source code, dependencies, architecture, test instructions, results, references, known limitations
References	Nov 1, 2026	Official documentation and papers for the frameworks actually used

Proposed video structure (not an official format): 0:00 to 0:30 problem; 0:30 to 1:15 environment; 1:15 to 2:15 AI architecture; 2:15 to 3:45 live demo; 3:45 to 4:30 measured results; 4:30 to 5:00 business value and next steps.

23. Immediate Action Checklist

- [ ] Register for the Innovation Lab
- [ ] Confirm team membership
- [ ] Connect GitHub to the event workspace
- [ ] Access Dataset 3 materials
- [ ] Download or open the digital twin
- [ ] Identify the available camera files
- [ ] Verify the robot controller interface
- [ ] Select the proposed technical approach
- [ ] Discuss feasibility with mentors
- [ ] Submit the team plan before October 14
- [ ] Build the initial baseline
- [ ] Measure results
- [ ] Prepare the final submission
- [ ] Practice the final presentation

24. Questions for Mentors

1. What exact FANUC robot model and controller are provided?
2. What format is the digital twin supplied in (USD, URDF, other)?
3. Is the robot controllable through Isaac Sim?
4. Are camera calibration parameters available?
5. What robot control interfaces are supported (for example, PLC, socket, or ROS 2 driver)?
6. Can teams run supervised real-robot tests?
7. Which objects must the system handle?
8. Are sorting destinations predefined?
9. Are there restrictions on external or pre-trained AI models?
10. What are the exact team plan form requirements?

25. Final Project Direction

AdaptiveVision will begin with a vision-guided pick-and-place baseline. The team will first establish reliable object detection, position estimation, and simulated robot control in the digital twin, and measure them against a fixed-coordinate program. Once the baseline is validated, the team may add adaptive grasp selection or failure recovery. The main objective is to demonstrate, with honest measurements, how Physical AI can make robotic automation more flexible in high-mix, low-volume manufacturing.

References

1. NVIDIA Isaac Sim documentation. docs.isaacsim.omniverse.nvidia.com
2. NVIDIA Isaac Lab documentation. isaac-sim.github.io/IsaacLab
3. OpenUSD documentation. openusd.org
4. OpenCV camera calibration tutorial. docs.opencv.org
5. Ultralytics YOLO documentation. docs.ultralytics.com
6. PyTorch documentation. pytorch.org/docs
7. ROS 2 documentation. docs.ros.org; MoveIt 2 documentation. moveit.picknik.ai
8. NVIDIA Cosmos. nvidia.com/en-us/ai/cosmos
9. Tobin, J. et al. (2017). Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. IROS 2017. arXiv:1703.06907